

Q-KET

장애 대응 표준 문서

Incident Response & Troubleshooting · TOP 10

프로젝트 장애 36건 중 실제 피해 범위와 재발 위험을 기준으로 선별한 핵심 10건

전체 다운 · 보안 노출 · 매출 직결 · 구조적 재발 위험을 우선 기준으로 정리하고, 원인/해결/상태뿐 아니라 현장에서 바로 확인할 수 있는 대응 및 검증 명령어 예시까지 표준화했습니다.

10

핵심 장애

3

긴급

5

높음

2

중간

02 장애 Top 10 요약

심각도와 영향 범위를 한눈에 보고, 상세 페이지로 바로 이어질 수 있도록 정리했습니다.

#	심각도	분류	장애	상태
1	긴급	보안	관리자 API 인가 우회	해결
2	긴급	가용성	10,000 명 부하 시 ALB 헬스체크 연쇄 장애	코드 반영 완료 · 1 만 명 규모 재검증 미완료
3	긴급	보안	NetworkPolicy 미집행	해결 방향 확립
4	높음	인프라 전반	destroy 순서 위반	해결 · 구조적 재발 방지 완료
5	높음	교차 서비스	부하테스트 중 ArgoCD·Grafana 동반 마비	해결
6	높음	모니터링	AMP 데이터소스 연결 실패	해결
7	높음	용량	KEDA 오토스케일링 완전 먹통	해결
8	높음	스케줄링	KEDA 스케일업 콜드스타트 CPU 경합	해결 · 재검증 완료
9	중간	비즈니스 로직	대기열 처리 속도 병목	해결
10	중간	구조적	EKS destroy 시 K8s addon 정리 실패	구조적 해법 확립 · 신규 addon 추가 시 재검토

핵심 인사이트

보안

“정책이 존재한다”와 “정책이 실제 집행된다”는 다름. API 인가와 NetworkPolicy 모두 실행작 검증이 필수.

가용성

헬스체크 경로는 사용자 요청 경로와 분리하고, 관측 엔드포인트 역시 일반 요청과 자원을 분리해야 함.

스케일링

Pod 스케일링만으로는 부족. Node 준비 시간과 Pod 분산 규칙을 함께 설계해야 급격한 트래픽을 견딜 수 있음.

IaC

K8s addon 과 AWS 리소스의 생명주기를 분리하지 않으면 destroy 시 finalizer·webhook 이 구조적으로 문제를 만들 수 있음.

02 상세 장애 분석 01-02

증상 → 원인 → 해결 → 상태 → 대응/검증 명령어 순서로 동일한 템플릿을 적용했습니다.

01 관리자 API 인가 우회

긴급 · 보안

전체 관리자 데이터 무방비 노출

증상 로그인 없이 회원·카테고리·예약이력 등 관리자 API 전체를 조회·수정·삭제할 수 있었음.

원인 AdminAccessInterceptor가 getRequestURI()를 사용해 context-path(/api)까지 포함된 값을 받았고, "/admin" 접두어 매칭이 성립하지 않아 관리자 인가 검사가 우회됨.

해결 getServletPath()로 교체(backend#41).

상태 해결

장애 대응 · 검증 명령어 예시

```
# 인증 없이 관리자 API 접근 재현
curl -i https://<HOST>/api/admin/<endpoint>
```

```
# 수정 후 기대 결과: 401 또는 403 확인
curl -i https://<HOST>/api/admin/<endpoint>
```

※ 명령어는 운영 환경에 맞게 namespace/host/resource명을 치환해 사용

02 10,000명 부하 시 ALB 헬스체크 연쇄 장애

긴급 · 가용성

HealthyHostCount=0 → 서비스 전체 다운

증상 과부하 시 파드가 헬스체크에 제때 응답하지 못해 ALB가 파드를 순차 제외했고, HealthyHostCount=0이 되면서 요청 100% 실패가 반복 재현됨.

원인 frontend 헬스체크가 무거운 SSR 페이지와 같은 경로를 사용했고, backend actuator가 일반 요청과 스레드풀을 공유함. SSR 캐싱도 적용되지 않아 부하가 가중됨.

해결 /healthz 전용 엔드포인트 생성, actuator 8081 포트 분리, SSR revalidate 캐싱 추가.

상태 코드 반영 완료 · 1만 명 규모 재검증 미완료

장애 대응 · 검증 명령어 예시

```
# 파드/헬스 상태 실시간 확인
kubectl get pods -A -w
```

```
# 전용 헬스 엔드포인트 확인
curl -i https://<HOST>/healthz
curl -i http://<BACKEND>:8081/actuator/health
```

※ 명령어는 운영 환경에 맞게 namespace/host/resource명을 치환해 사용

03 상세 장애 분석 03-04

증상 → 원인 → 해결 → 상태 → 대응/검증 명령어 순서로 동일한 템플릿을 적용했습니다.

03 NetworkPolicy 미집행

긴급 · 보안

정책은 존재했지만 3주간 실제 트래픽 차단 미작동

증상 backend-allow-same-namespace 정책이 kubectl 조회에는 존재했지만, 다른 네임스페이스에서 backend로 curl 요청이 그대로 통과함.

원인 aws-eks-nodeagent는 있었지만 NetworkPolicy를 PolicyEndpoint CR로 변환하는 amazon-network-policy-controller-k8s가 없어 규칙이 에이전트에 전달되지 않음.

해결 vpc-cni를 EKS 관리형 addon으로 등록하고 enableNetworkPolicy=true 적용. 적용 전 ALB 소스 CIDR을 ipBlock으로 선택용해야 전체 차단 사고를 피할 수 있음.

상태 해결 방향 확립

장애 대응 · 검증 명령어 예시

```
kubectl get networkpolicy -A
kubectl get policyendpoint -A
kubectl get ds -n kube-system aws-node
```

```
# 다른 namespace에서 차단 여부 검증
kubectl exec -n <OTHER_NS> <POD> -- curl -m3
http://<BACKEND>
```

※ 명령어는 운영 환경에 맞게 namespace/host/resource명을 치환해 사용

04 destroy 순서 위반

높음 · 인프라 전반

IAM 손상 + 고아 리소스 + 클러스터 삭제 불가

증상 야간 destroy 순서가 꼬이며 IAM 권한 손상, ALB·보안그룹 고아 리소스, admission webhook/finalizer 대기 때문에 클러스터 자체가 삭제되지 않음.

원인 순수 AWS 리소스와 K8s addon(Helm) 리소스가 같은 Terraform state에 있어 destroy 순서를 보장하기 어려웠음.

해결 수동 복구 후 01_infrastructure(AWS) / 02_k8s-addon(K8s) 레이어 분리.

상태 해결 · 구조적 재발 방지 완료

장애 대응 · 검증 명령어 예시

```
# 표준 destroy 순서
cd 02_k8s-addon &&terraform destroy
cd ../01_infrastructure &&terraform destroy
```

```
# 잔여 리소스 확인
terraformstate list
```

※ 명령어는 운영 환경에 맞게 namespace/host/resource명을 치환해 사용

04 상세 장애 분석 05-06

증상 → 원인 → 해결 → 상태 → 대응/검증 명령어 순서로 동일한 템플릿을 적용했습니다.

05 부하테스트 중 ArgoCD·Grafana 동반 마비

높음 · 교차 서비스

backend 부하가 관측/배포 도구까지 전파

증상 k6 로그인 1,000명 부하테스트 때 backend와 무관한 ArgoCD·Grafana까지 응답 불가.

원인 HikariCP 연결 풀이 과도했고 연결 생성이 CPU를 크게 소모함. 버스터블 워커 노드의 CPU 크레딧이 노드 단위로 공유되면서 같은 노드의 다른 파드까지 자원 고갈 영향을 받음.

해결 HikariCP maximum-pool-size 40 → 10으로 축소.

상태 해결

장애 대응 · 검증 명령어 예시

```
kubectl top nodes
kubectl top pods -A --sort-by=cpu
```

```
# 파드 배치 위치 확인
kubectl get pods -A -o wide
```

※ 명령어는 운영 환경에 맞게 namespace/host/resource명을 치환해 사용

06 AMP 데이터소스 연결 실패

높음 · 모니터링

checkHealth 통과 · 실제 PromQL만 403

증상 Grafana AMP 데이터소스 연결 확인은 통과하지만 실제 PromQL 쿼리는 403 Missing Authentication Token으로 실패.

원인 IRSA/access key 방식과 무관하게 동일하게 실패했고, 데이터소스 queryData 요청에 Authorization 헤더가 붙지 않는 정황을 확인. jsonData.sigV4Auth 필드 누락이 우회 포인트였음.

해결 서버 설정과 함께 데이터소스 jsonData에 sigV4Auth: true를 명시.

상태 해결

장애 대응 · 검증 명령어 예시

```
# Grafana datasource 설정 확인 포인트
jsonData.sigV4Auth: true
```

```
# 수정 후 PromQL 예시
up
sum(rate(http_server_requests_seconds_count[5m]))
```

※ 명령어는 운영 환경에 맞게 namespace/host/resource명을 치환해 사용

05 상세 장애 분석 07-08

증상 → 원인 → 해결 → 상태 → 대응/검증 명령어 순서로 동일한 템플릿을 적용했습니다.

07 KEDA 오토스케일링 완전 먹통

높음 · 용량

metrics-server 부재로 HPA TARGETS=<unknown>

증상 응답 지연과 CPU 스로틀링이 발생해도 backend replica가 4개에서 늘지 않았고 HPA TARGETS가 <unknown>으로 고정됨.

원인 KEDA CPU 트리거가 의존하는 metrics-server가 클러스터에 설치되어 있지 않았음.

해결 metrics-server를 Helm으로 설치(--kubelet-insecure-tls). 재테스트에서 replica 4 → 8 증가 확인.

상태 해결

장애 대응 · 검증 명령어 예시

```
kubectl get hpa -A -w
kubectl top pods -A
```

```
helm upgrade --install metrics-server metrics-server/
metrics-server \
  -n kube-system --create-namespace \
  --set args=(--kubelet-insecure-tls)
```

※ 명령어는 운영 환경에 맞게 namespace/host/resource명을 치환해 사용

08 KEDA 스케일업 콜드스타트 CPU 경험

높음 · 스케줄링

신규 파드가 한 노드에 몰리며 반복 재시작

증상 트래픽 급증 시 신규 파드가 여러 노드에 고르게 분산되지 않고 한 노드에 몰려 CPU가 포화되고 파드 재시작이 반복됨.

원인 cluster-autoscaler의 노드 준비 지연 + 강제 분산 규칙 부재 + 강제 분산 시 새 노드를 기다리는 지연이 서로 얹힌 구조적 문제.

해결 Karpenter 전환 + 오버프로비저닝 + topologySpreadConstraints의 DoNotSchedule 적용. 2,000명 재검증에서 요청 실패율 0%, p95 15.82초 기록.

상태 해결 · 재검증 완료

장애 대응 · 검증 명령어 예시

```
kubectl get pods -o wide -n <APP_NS>
kubectl get nodes -w
kubectl describe pod <POD> -n <APP_NS>
```

```
# 분산 규칙 확인
kubectl get deploy <BACKEND> -o yaml | grep -A20
topologySpreadConstraints
```

※ 명령어는 운영 환경에 맞게 namespace/host/resource명을 치환해 사용

06 상세 장애 분석 09-10

증상 → 원인 → 해결 → 상태 → 대응/검증 명령어 순서로 동일한 템플릿을 적용했습니다.

09 대기열 처리 속도 병목

중간 · 비즈니스 로직

2,000명 테스트 내내 대기열 통과 0명

증상 2,000명 규모 부하테스트에서 테스트 종료까지 대기열을 벗어나 예매 화면으로 진입한 사용자가 0명.

원인 MAX_ACTIVE_USERS=10이 너무 작아 처리량이 초당 극소수 수준에 그침.

해결 MAX_ACTIVE_USERS를 150으로 상향.

상태 해결

장애 대응 · 검증 명령어 예시

```
# 현재 환경변수 확인
kubectl exec deploy/ <BACKEND> -- printenv
| grep MAX_ACTIVE_USERS

# 배포 설정 반영 후 값 확인
kubectl get deploy <BACKEND> -o yaml
| grep -A2 MAX_ACTIVE_USERS
```

※ 명령어는 운영 환경에 맞게 namespace/host/resource명을 치환해 사용

10 EKS destroy 시 K8s addon 정리 실패

중간 · 구조적

finalizer 처리 주체 소실로 반복되는 구조적 리스크

증상 EBS CSI, KEDA, GatewayClass 등에서 4번과 같은 패턴이 반복될 가능성이 있었고, 매일 destroy마다 새로운 addon에서 재발할 수 있는 구조였음.

원인 K8s addon 리소스가 해당 컨트롤러보다 먼저 삭제되면 finalizer를 처리할 주체가 사라져 리소스가 영구 대기 상태에 빠짐.

해결 Layer 1(AWS) / Layer 2(K8s addon) 분리를 표준화하고 신규 addon마다 depends_on 및 destroy 순서를 재검토.

상태 구조적 해법 확립 · 신규 addon 추가 시 재검토

장애 대응 · 검증 명령어 예시

```
# 삭제 전 addon 계층 정리
cd 02_k8s-addon && terraform destroy

# Terminating/finalizer 잔여 확인
kubectl get all -A
kubectl get gatewayclass, keda, scaledobject -A 2>/dev/null || true
```

※ 명령어는 운영 환경에 맞게 namespace/host/resource명을 치환해 사용